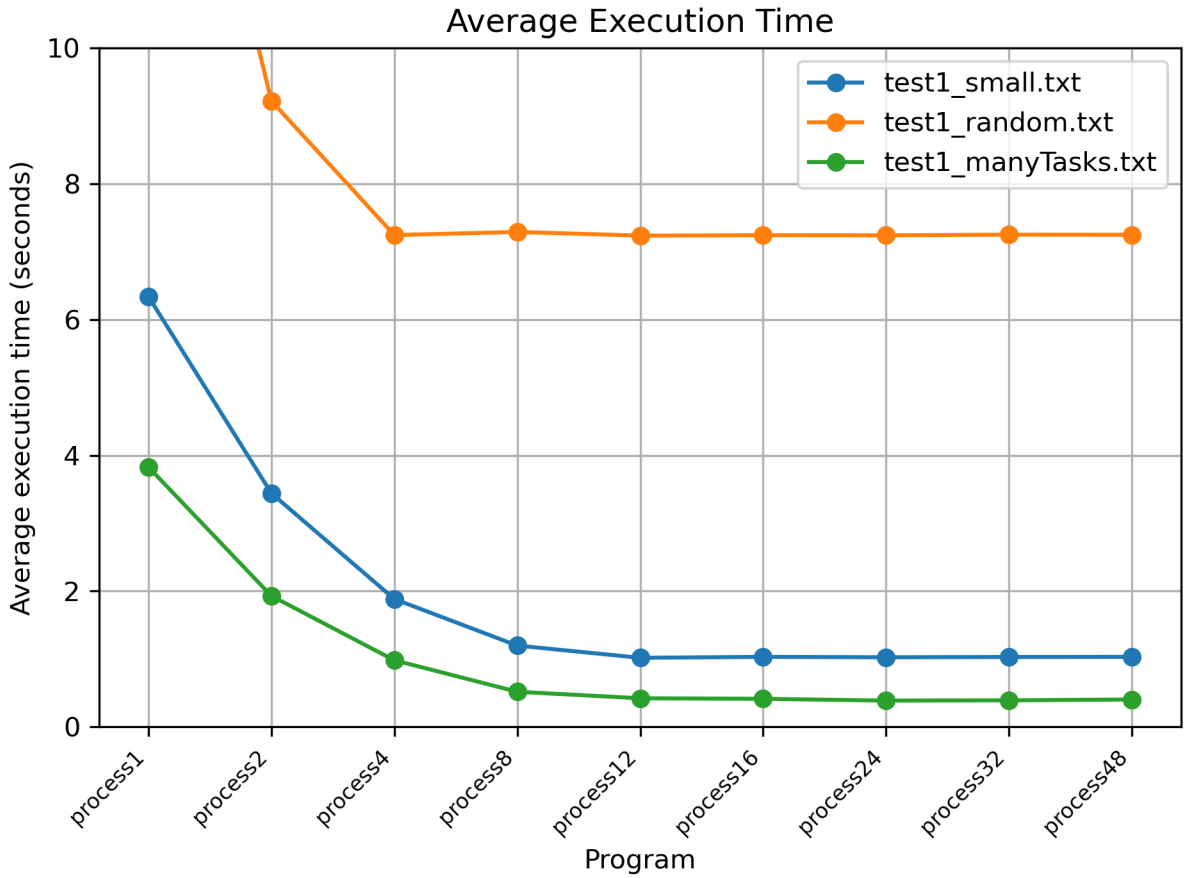
ENCE360 Assignment
Jamie Boyd
67354831

For my first experiment I've split it into two parts as I wanted to test both how thread.c and process.c scaled if I increased their respective defined values of the number of threads and number of children. I have used the same test input files for these two graphs: test1_small is a small number of queries that are low slice amounts, test1_manyTasks is a large number of queries with small slices, and test1_random are tasks with differening amounts of slices.

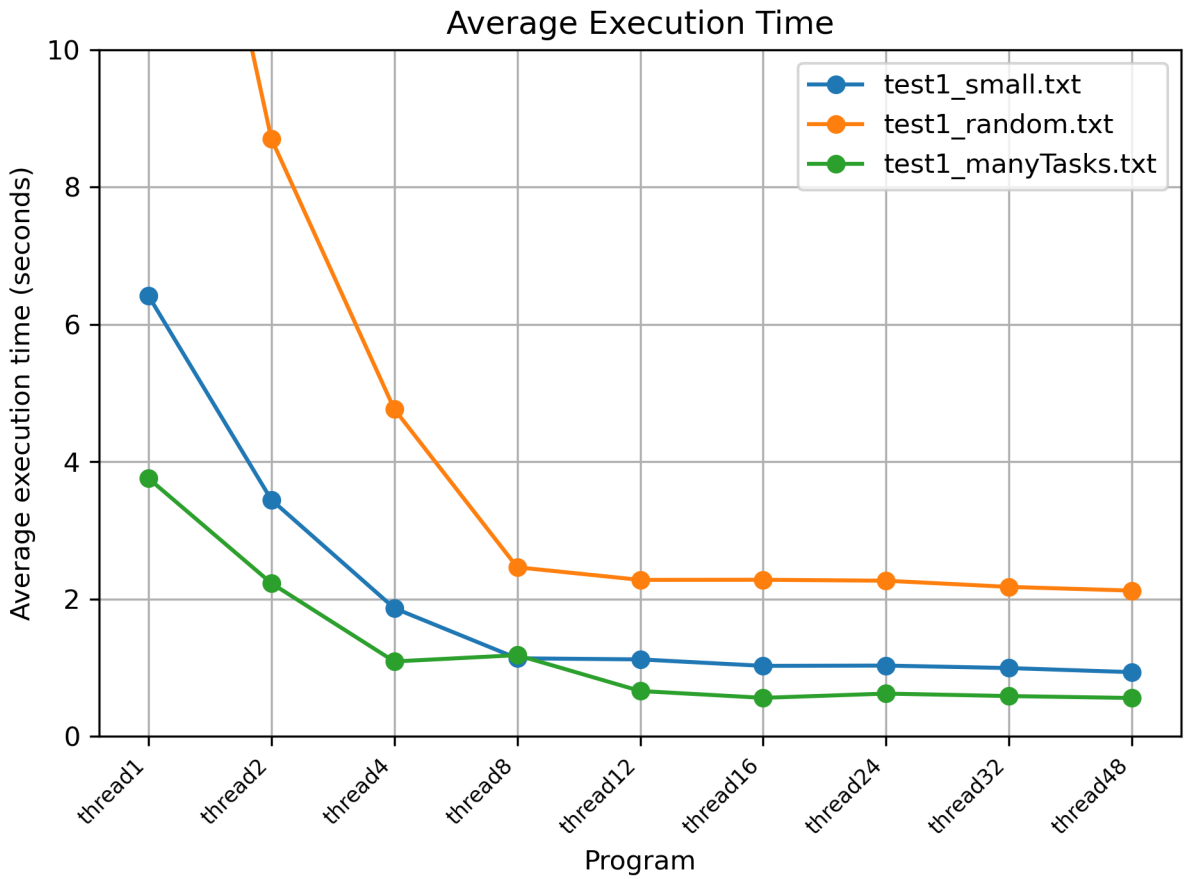
Experiment 1:
This graph is limited at 10 so that it is easier to read, the process1 time for the random test is approximately 16.7 seconds.



What can be seen in this graph is that increasing the maximum number of children has a significant increase in time, with random getting to just less than half, while manyTasks and small are reduced to taking significnatly less time.
If we look specifically at random it caps out at 4 processes, which is as expected because the test file is written to include 2 queries that are significantly larger than the rest. This then means that if there are 2 or less processes running simultaneously, they will both get stuck on the large queries. However, if there are more processes available, two will get stuck on the large queries while the rest will have already completed everything else, even if there is only one extra process. This is an example of Ahmdal's law which explains that the portion of a program that cannot be parallelised limits the maximum speedup of the program [2].
We have a different interaction with small and manyTasks, because they don't have outlier queries. In these tests we can see that the time trends towards 12 processes. This can be explained by the design of the cpu, which has 12 cores: 8 performance cores and 4 efficiency cores, that make up a

total of 20 threads [1]. When the operating system has to assign more than 8 processes, it either has to use one of the secondary threads on a performance core, or an efficiency core. These two options are going to make the process slower than if it had full control of a performance core, and so we can see how there is a minute decrease in time taken from 8 to 12 processes, but past that point it is about the same. One factor to consider when thinking about this is that vscodium was open on the machine to run the programs, so that would potentially be taking up cpu resources aswell.

Experiment 2:
This graph is limited at 10 so that it is easier to read, the thread1 time for the random test is approximately 16.9 seconds.

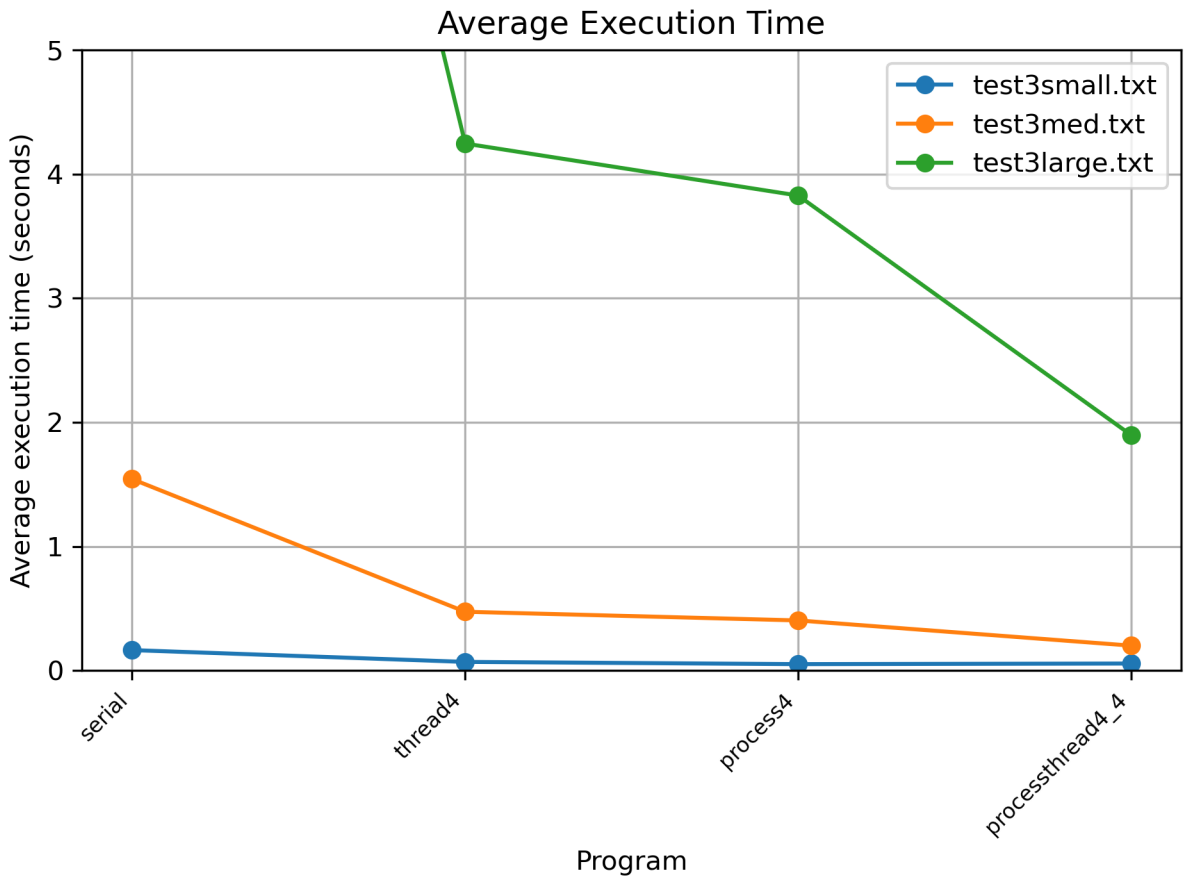


In this thread version of the experiment we can see that increasing the maximum number of threads being used also has a significant increase in time. However, compared to using children, there is no difference in behaviour for the random test. This highlights the advantage that threads have in this test scenario, because instead of solving one query per process, they split each query up and solve it simultaneously with all the threads combined. This allows threads an advantage when working with skewed queries as they split up each integration within each query instead of splitting up the total queries.
If we look at all 3 of the tests we can see that they all generally trend towards 12 threads being the limit, with some variance at higher numbers of threads between each test. This again can be explained partly by the arrangment of the cpu threads available, with up to 8 threads being available as using a singular performance core, and past this point they are either running on an efficiency core or the second thread of a performance core [1]. However this isn't the only thing effecting the times for the higher thread counts. Other things to consider with more threads is that there is higher overhead for creation and destruction, so with all 3 tests as there are more and more threads they get

diminishing returns on time reduction. One last thing to consider is that there will be time spent context switching for thread counts larger than at least 20 (potentially even starting at 19 or lower because vscode was open to run the tests) that also add overhead to the time.

Experiment 3:

This experiment is testing the effect of increasing the number of slices. Each input test has the same number of queries of the same types and ranges, but the number of slices is multiplied by 10 each time. The first value of serial for the large test is at approximately 15.4 seconds, so I decided to leave it off the graph so that we can get a closer view.



The graph shows that execution time increased with the workload for all the implementations, and that the parrallel programs all consistently performed faster than the serial program. Every program experienced approximately a 9 – 10 times increase in time when going from teh medium test to the large test, but it was not as proportional of an increase if looking from the small test to the medium. This indicates that with larger inputs of slices the time to execute the queries becomes proportional to the amount of integration required.
The combination of using children and threads shows a smaller increase in time from small to medium, which could indicate that a large amount of the time that processthread spent for the small test was due to overhead, as creating threads already has a noticable overhead for small tasks [3].
If we now compare the difference between the serial time for the large input and the processthread time, we can see that it is around 8 times quicker. This clearly shows that parallelising a non trivial task has immense benefits, and that the computational overhead of doing so has no real impact on the final execution time.

References:
[1] Intel Corporation, "Intel® Core™ i7-12700 Processor (25M Cache, up to 4.90 GHz) Specifications," *Intel Product Specifications*, 2022. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/134591/intel-core-i712700-processor-25m-cache-up-to-4-90-ghz/specifications.html. [Accessed: Oct. 9, 2026].

[2] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017, pp. 50.

[3] "pthread_create(3p) — Linux manual page," IEEE Std 1003.1-2017, Standard for Information Technology -- Portable Operating System Interface (POSIX), The Open Group Base Specifications Issue 7, 2018 Edition. [Online]. Available: https://man7.org/linux/man-pages/man3/pthread_create.3p.html. [Accessed: Oct. 9, 2026].